



POLITECHNIKA KRAKOWSKA im. T. Kościuszki
Wydział Informatyki i Telekomunikacji

Techniki Projektowania Frontendowego

Raport rozbieżności między konspektem a implementacją Portfolio fotografa

Radosław Karpiński, Marcin Klimek, Paweł Mączałowski

Prowadzący:
dr hab. inż. arch. Paweł Ozimek
mgr inż. Adrian Widlak

Kraków, 10 czerwca 2026

Spis treści

1	Cel i zakres raportu	1
1.1	Materiały źródłowe	1
2	Podsumowanie ogólne	1
3	Główne rozbieżności	1
3.1	Struktura serwisu i architektura informacji	1
3.2	Portfolio i media	2
3.3	Kontakt i konwersja	2
3.4	Wielojęzyczność	2
3.5	Design	2
3.6	Stos technologiczny	3
3.7	Wymagania niefunkcjonalne	3
3.8	Elementy zgodne z konspektem	3
4	Rozbieżności względem projektu w Figmie	3
4.1	Nawigacja	3
4.2	Układ strony głównej	4
4.3	Portfolio manager	4
5	Dodatki w finalnej implementacji względem konspektu	4
6	Wnioski	4

1 Cel i zakres raportu

Celem raportu jest opisanie rozbieżności między zaplanowaną wersją projektu, opisaną w konspekcie „Portfolio fotografa”, a faktyczną implementacją znajdującą się w repozytorium `tpf-projekt-portfolio`.

Analiza obejmuje zarówno zmiany funkcjonalne, jak i różnice w architekturze informacji, modelu treści, technologii, warstwie wizualnej oraz wymaganiach niefunkcjonalnych. Raport nie ocenia, czy każda zmiana była błędem projektowym. Część zmian wynika z ograniczenia zakresu, część z celowej zmiany kierunku technicznego, a część z niedokończenia elementów zaplanowanych w konspekcie.

1.1 Materiały źródłowe

Porównanie wykonano na podstawie:

- konspektu projektu dostarczonego w pliku PDF,
- projektu przygotowanego w Figmie,
- dodatkowej listy różnic przygotowanej przez drugiego agenta,
- inspekcji kodu źródłowego bieżącej aplikacji.

2 Podsumowanie ogólne

Finalna implementacja jest znacznie węższa od wersji planowanej. Konspekt zakładał rozbudowany serwis portfolio fotografa i filmowca, z osobnymi obszarami dla filmu i fotografii, podstronami informacyjnymi, treściami ofertowymi, kalendarzem, obsługą wielu języków, osadzonym wideo, filtrowaniem, opisami projektów oraz headless CMS. Finalny produkt jest natomiast prostą aplikacją SPA z widokiem portfolio, stroną kontaktu oraz własnym panelem administracyjnym opartym o Firebase.

Największe zmiany:

- struktura informacyjna została uproszczona do dwóch publicznych widoków: `Work` i `Contact`,
- materiały wideo nie są odtwarzane ani osadzone na stronie,
- portfolio nie ma filtrów, osobnych stron projektów, opisów, metadanych ani case studies,
- kontakt nie realizuje wysyłki formularza i nie ma kalendarza,
- nie zaimplementowano wielojęzyczności,
- stack technologiczny zmienił się z `Next.js` + `CMS` + `PostgreSQL` + `Vercel` na `Vite SPA` + `Firebase` + `Cloudflare Workers`,
- `CMS` został zastąpiony własnym panelem administracyjnym.

3 Główne rozbieżności

3.1 Struktura serwisu i architektura informacji

Największa zmiana dotyczy zakresu całej witryny. Konspekt opisywał pełny serwis portfolio z osobnymi obszarami dla filmu, fotografii, informacji o autorze, oferty i kontaktu. W implementacji publiczna część została ograniczona do widoku portfolio oraz strony kontaktowej. Nie powstały osobne podstrony dla filmu, fotografii, oferty ani sekcji „O mnie”.

Zmianie uległa także ścieżka użytkownika. Plan zakładał, że strona główna będzie prowadzić przez najważniejsze obszary serwisu i kończyć się kontaktem lub kalendarzem. Finalna wersja działa bardziej

jak prosta galeria: hero kieruje do zewnętrznego showreela albo do siatki projektów, a kontakt jest osobnym widokiem dostępnym z nawigacji.

Nie zaimplementowano również stron szczegółowych projektów. W konspekcie projekt miał mieć własną stronę z odtwarzaczem, opisem i galerią. W finalnym produkcie kliknięcie projektu otwiera lightbox z pojedynczym obrazem, tytułem i kategorią.

3.2 Portfolio i media

Konspekt traktował wideo jako jeden z głównych typów treści. Zakładano osadzanie materiałów z YouTube lub Vimeo, odtwarzanie ich bezpośrednio na stronie, osobną prezentację dłuższych form i rolek oraz siatkę typu masonry. Finalna implementacja nie zawiera odtwarzacza wideo ani embedów. Projekty są reprezentowane przez pojedynczy obraz zapisany w polu `imageUrl`, a showreel jest tylko zewnętrznym linkiem.

Portfolio zostało też uproszczone pod względem organizacji treści. Kategorie istnieją jako etykiety tekstowe, ale nie ma filtrów, tagów ani osobnego podziału na film i fotografię. Pierwszy projekt jest wyróżniony wizualnie, jednak nie wynika to z osobnego pola w modelu danych, tylko z pozycji na liście.

Zakres danych o projektach jest dużo mniejszy niż planowano. Konspekt oraz badanie użytkowników wskazywały opisy, metainformacje, case studies, referencje i materiały dodatkowe jako wartościowe elementy portfolio. W finalnym modelu projektu pozostały tylko podstawowe pola: tytuł, kategoria, obraz i orientacja.

3.3 Kontakt i konwersja

Planowana strona kontaktu miała być miejscem finalizacji ścieżki klienta: formularzem, danymi kontaktowymi i kalendarzem do umawiania rozmów. W implementacji widoczne są dane kontaktowe oraz formularz, ale formularz nie wysyła wiadomości, ponieważ obsługa submitu jedynie zatrzymuje domyślne przeładowanie strony. Nie ma też kalendarza ani integracji do rezerwacji terminów.

Nie powstała również sekcja cen lub pakietów, mimo że w badaniu preferencji użytkowników pojawiła się potrzeba orientacyjnej informacji o kosztach. Finalna wersja wspiera więc głównie kontakt przez e-mail i linki zewnętrzne.

3.4 Wielojęzyczność

Konspekt zakładał obsługę języka polskiego i angielskiego, automatyczne wykrywanie języka użytkownika oraz ręczny przełącznik. Finalna aplikacja działa w jednym wariancie językowym. Model treści nie przewiduje wersji PL/EN, nie ma plików tłumaczeń ani przełącznika języka w interfejsie.

3.5 Design

Kierunek minimalistyczny i nowoczesny został w ogólnym sensie zachowany, ale finalna warstwa wizualna nie trzyma się dokładnie palety i typografii zapisanych w konspekcie. Zmieniono odcienie kolorów, nie użyto Roboto, a Montserrat nie pełni roli fontu nagłówkowego. Zamiast rozbudowanych animacji są głównie podstawowe przejścia i efekty hover.

Różnice projektowe są mniej istotne niż różnice zakresowe, bo ogólny charakter wizualny pozostaje zbliżony do planu. Największym ograniczeniem jest raczej brak wielu zaplanowanych widoków, przez co nie powstał pełny system wizualny dla ofert, case studies, stron projektów czy osobnych sekcji portfolio.

3.6 Stos technologiczny

Stack technologiczny został istotnie zmieniony. Konspekt wskazywał rozwiązanie oparte o Next.js, React, TypeScript, SSR, headless CMS, PostgreSQL i wdrożenie na Vercel. Finalna implementacja jest aplikacją Vite SPA z React Routerem, Firebaseem i konfiguracją Cloudflare Workers.

Największą konsekwencją tej zmiany jest odejście od klasycznego headless CMS. Zamiast Strapi, Sanity lub Contentful powstał własny panel administracyjny z logowaniem przez Firebase Auth oraz danymi w Firestore i Storage. Panel pozwala edytować część treści, ale nie obejmuje pełnej struktury zaplanowanej w konspekcie, ponieważ nie ma pól dla oferty, sekcji „O mnie”, opisów projektów, wielojęzyczności ani rozbudowanych danych portfolio.

Z planu technologicznego zachowano React, TypeScript, pnpm i Tailwind CSS. Nie użyto natomiast styled-components ani PostgreSQL.

3.7 Wymagania niefunkcjonalne

Responsywność została w dużej mierze zrealizowana, natomiast SEO i optymalizacja zasobów są skromniejsze niż zakładał konspekt. Aplikacja jest SPA bez SSR, ma statyczny tytuł i opis meta, ale nie zawiera pełnego zestawu rozwiązań SEO, takich jak Open Graph, sitemap, dane strukturalne czy dynamiczne metadane podstron.

Optymalizacja obrazów została wykonana częściowo: obrazy są kompresowane przy uploadzie, ale przy renderowaniu nie ma responsywnych wariantów srcset ani nowoczesnych formatów zależnych od urządzenia. Dostępność również jest częściowa. Lightbox ma podstawowe atrybuty ARIA i obsługę klawiatury, ale brak bardziej kompletnej obsługi, takiej jak pułapka fokusu czy skip link.

3.8 Elementy zgodne z konspektem

Dla pełnego obrazu warto odnotować również elementy, które nie są rozbieżnościami albo są realizacją częściową:

- aplikacja jest napisana w React i TypeScript,
- użyto pnpm,
- użyto Tailwind CSS,
- istnieje responsywny układ strony,
- istnieje lightbox z obsługą klawiatury,
- widoczny jest adres e-mail,
- linki społecznościowe są edytowalne z panelu administracyjnego,
- istnieje mechanizm samodzielnej edycji części treści przez klienta,
- estetyka jest zasadniczo minimalistyczna i nowoczesna.

4 Rozbieżności względem projektu w Figmie

Oprócz zmian względem pierwotnego konspektu pojawiły się również różnice między finalną implementacją a projektem interfejsu przygotowanym w Figmie. Są to przede wszystkim zmiany w kompozycji głównych widoków oraz w sposobie rozwiązywania panelu administracyjnego.

4.1 Nawigacja

Navbar w finalnej wersji nie odtwarza dokładnie układu z Figmy. W makiecie pasek nawigacji zawierał logo po lewej stronie, linki Work, Contact i Admin w środkowej części oraz przycisk View Reel po prawej

stronie. Implementacja zachowała logo oraz podstawowe linki publiczne, ale usunęła z paska link Admin i przycisk View Reel.

W efekcie navbar jest prostszy i bardziej przystosowany pod regularnego użytkownika. Część administracyjna nie jest eksponowana w głównej nawigacji, a akcja związana z obejrzeniem showreela została przeniesiona do sekcji hero zamiast pozostać stale dostępna w górnym pasku.

4.2 Układ strony głównej

Strona główna używa innego układu zdjęć niż projekt w Figmie. W makiecie portfolio pod hero ma bardziej swobodny, asymetryczny układ: duże zdjęcie poziome zaczyna sekcję po lewej stronie, obok znajduje się pionowy portret, a kolejne fotografie są rozmieszczone z dużymi przesunięciami i nieregularnym rytmem. Całość przypomina bardziej kompozycję redakcyjną lub masonry niż klasyczną listę kart.

W implementacji portfolio jest bardziej uporządkowane. Pierwszy projekt jest pokazany jako pojedyncza, wyśrodkowana karta z podpisem i kategorią, a kolejne projekty tworzą regularniejszą dwukolumnową siatkę kart. Każdy element ma czytelny tytuł i metadane pod obrazem. Zmiana poprawia jednoznaczność listy projektów, ale oddala stronę od bardziej luźnej, wizualnej kompozycji zaprojektowanej w Figmie.

4.3 Portfolio manager

Największa różnica względem Figmy dotyczy panelu zarządzania portfolio. Portfolio manager w finalnej aplikacji jest zbudowany zupełnie inaczej niż w makiecie. Różni się strukturą widoku, układem formularzy, sposobem prezentowania listy projektów oraz ogólnym charakterem interfejsu.

Zmiana ta jest istotna, ponieważ portfolio manager jest osobnym narzędziem pracy, a nie tylko detalem wizualnym strony publicznej. Finalna wersja realizuje podstawowy cel, czyli edycję treści i dodawanie projektów, ale nie zachowuje zaprojektowanego w Figmie sposobu zarządzania portfolio.

5 Dodatki w finalnej implementacji względem konspektu

Finalna aplikacja zawiera również kilka elementów, których konspekt nie zakładał wprost. Najważniejsze z nich to:

- własny panel administracyjny z logowaniem przez Firebase Auth,
- przechowywanie treści i obrazów w usługach Firebase,
- wdrożenie przygotowane pod Cloudflare Workers,
- integracje analityczne, m.in. Google Analytics, Hotjar i Contentsquare,
- kompresja obrazów po stronie przeglądarki przed wysłaniem do Storage,
- obsługa klawiatury w lightboxie.

6 Wnioski

Finalny produkt zachowuje rdzeń idei portfolio wizualnego, ale jest znacząco uproszczoną wersją względem konspektu. Największa zmiana dotyczy przesunięcia projektu z pełnego serwisu portfolio fotografa i filmowca w stronę prostej galerii obrazów z administracją treści. W konsekwencji zrealizowano przede wszystkim podstawową prezentację prac, kontakt e-mail, responsywność oraz częściową edycję treści, natomiast pominięto większość funkcji budujących rozbudowane doświadczenie portfolio:

wideo, osobne obszary działalności, opisy projektów, filtrowanie, ofertę, sekcję o autorze, kalendarz, wielojęzyczność i pełniejsze SEO.

Najważniejsze rozbieżności mają charakter nie tylko kosmetyczny, ale zakresowy i architektoniczny. Zmiana z Next.js/SSR/headless CMS/PostgreSQL na Vite SPA/Firebase/Cloudflare Workers wpływa na SEO, sposób zarządzania treścią, możliwości rozwoju i zgodność z pierwotną specyfikacją. Jednocześnie finalna implementacja ma kilka praktycznych dodatków, przede wszystkim własny panel administracyjny, Firebase Auth, Storage oraz analitykę, które nie były bezpośrednio opisane w konspekcie.